

NEURAL_{TEX}: a machine learning library written in pure L_{TEX}

James A. D. Gardner, Will Rowan and William A. P. Smith
Department of Computer Science
University of York, UK

Abstract—In this paper, we introduce NEURAL_{TEX}, which we believe to be the first deep learning library written entirely in L_{TEX}. As part of your L_{TEX} document you can specify the architecture of a neural network and its loss functions, define how to generate or load training data, and specify training hyperparameters and experiments. When the document is compiled, the L_{TEX} compiler will generate or load training data, train the network, run experiments, and generate figures. This paper generates a random 100 point spiral dataset, trains a two layer MLP on it, evaluates on a different random spiral dataset, produces plots and tables of results. The paper took 48 hours to compile and the entire source code for NEURAL_{TEX} is contained within the source code of the paper.

We propose two new metrics: the Written In Latex (WIL) metric measures the proportion of a machine learning library that is written in pure L_{TEX}, while the Source Code Of Method in Source Code of Paper (SCOMISCOP) metric measures the proportion of a paper’s implementation that is contained within the paper source. We are state-of-the-art for both metrics, outperforming the ResNet and Transformer papers, as well as the PyTorch and Tensorflow libraries. Source code, documentation, videos, crypto scams and an invitation to invest in the commercialisation of NEURAL_{TEX} are available at neuralatex.com.

I. INTRODUCTION

While often used as a document preparation markup language, in fact L_{TEX} is itself a Turing-complete programming language. L_{TEX} natively provides variables, loops and conditionals via T_{EX} primitives, while other features such as object-orientation are supported via appropriate packages [10]. Yet, if you consult the curated list of awesome machine learning frameworks, libraries and software (by language) [7], L_{TEX} is not even included as a category of programming language. This motivated us to write a deep learning library entirely in L_{TEX}.

```
\expanded{
  \noexpand\pgfoonew\expandafter\noexpand
  \csname #2\endcsname=
  new Value(\newdata,{\selfid,\otherid
    },*,0)
}
```

Fig. 1: A pure L_{TEX} implementation has the benefit of high code readability. For example, returning the result of a product between two value objects only uses the word ‘expand’ four times.

A. Wait, what?

When we talk about ‘compiling’ a L_{TEX} document, really what we mean is that the program written in your L_{TEX} document is executed. The L_{TEX} engine parses and processes the document, resolving macros, executing conditionals, and formatting content before outputting the final result. NEURAL_{TEX} is a collection of .tex L_{TEX} source files that you can include in your document via `\input{...}`. These provide a series of commands that enable you to define a neural network, load or generate training and evaluation data, train the network via backprop and run inference on the trained network. All of this happens when your document is ‘compiled’. The results shown in Section IV are all dynamically generated at compile time using a neural network that was itself trained while the L_{TEX} document compiled.

B. Why L_{TEX}?

1) *An ideal programming language:* Do you sometimes find it hard to decide whether to expand, not expand or expand after? Well, with L_{TEX} you can do all of those and more (see Figure 1). Who needs variables when you have macros? Who needs arrays when you can create comma, separated, strings? Who wants a simple for loop when you can have `pgfplotsforeachungrouped`?

2) *The self-contained paper*: Machine learning is facing a reproducibility crisis. Too often “code will be made available upon paper acceptance” becomes a GitHub repository that is empty but for a README.md containing words that strike fear into the hearts of PhD students hoping for an easy comparative evaluation: “Code coming soon”.

On the other hand, any machine learning researcher worth their salt uploads all of their papers to arXiv. The requirement by arXiv for all papers to submit the full $\LaTeX$ source necessary to compile the paper presents an opportunity. In this paper, we exploit the universality of access to paper source files as a solution to the reproducibility problem. We call this the *self-contained paper*. A self-contained paper must contain within its $\LaTeX$ source all training data, implementation of the method, and experiments in a form that can be run by the $\LaTeX$ compiler itself. The training of any models therefore takes place as part of the compilation of the paper. Since arXiv makes the paper source files available, having access to the paper is equivalent to having access to the code. No more “code coming soon”!

3) *Additional benefits*: First, overleaf becomes more than just a cloud-based $\LaTeX$ editor. It is now also your (free) cloud compute service. Second, since arXiv limits the size of any submission to 50MB, researchers are forced to work on very small datasets and models. This helps reduce the unfair advantage industry has in accessing large GPU resources. Third, $\LaTeX$ is a programming language, so why have to context switch between your paper source and Python IDE - just do everything in one place ($\LaTeX$)! Finally, do you always forget git commands? From now on, your code link can simply point to the paper source files on arXiv. No need for a GitHub repository (who needs version control anyway?).

C. Why?

More seriously, implementing a neural network library in such an awkward ‘programming language’ has been an incredible learning experience. You might think you understand backprop, but actually implementing it from scratch in a programming language that lacks most of the basic features you rely on in any other language is a seriously fun and intellectually challenging exercise. It’s also worth emphasising that none of the authors are particularly knowledgeable about $\LaTeX$. It’s quite possible that we made life much harder for ourselves than necessary. We were never fully confident about the scope of macros, so we had to use defensive naming

conventions in case they were global. Another interesting challenge is that LLMs like ChatGPT are pretty terrible at programming in $\LaTeX$ so help is limited. If you ask them how to do something complicated in $\LaTeX$, they tend to politely suggest you use python instead and provide python code. Or if they do provide $\LaTeX$ code it often doesn’t work.

D. Related work

Important previous work has also considered implementation of different types of program in neglected languages. For example, Wildenhain [13] showed that MS PowerPoint is Turing-complete, providing a cross-platform, intuitive, GUI-based programming language in which any conceivable program could be implemented. Closely related to our concept of unifying both the implementation of a method with the source code of its write-up, Murphy [8] showed how a single file could simultaneously be both a valid executable file and also a plain text file containing the paper itself. Like us, Wildenhain [14] also understand the superiority of $\LaTeX$, but rather than make $\LaTeX$ more powerful, they dumb it down to a WYSIWYG (What You See Is Pretty Close To What Other Tools Can Get) editor, WordTeX. The most closely related previous work is ExcelNet [3] that implemented neural networks in Microsoft Excel. However, they did not implement backpropagation and only supported pretrained (or user edited) network weights. Inspiration for our catchy name came from ACTION [2]. We have not implemented AMOR [12] in the author ordering for this paper due to the already-heavy compile demands but its use can be assumed by imagining the author list randomly shuffling before your eyes.

II. IMPLEMENTATION

Our implementation is heavily based on micrograd [6], although with a better choice of implementation language. Like micrograd, $\text{Neural}\LaTeX$ implements backpropagation (reverse-mode autodiff) over a dynamically-constructed DAG which can implement arbitrarily complex neural networks. Unlike micrograd (which comprises around 150 lines of python), our autograd engine requires nearly 700 lines of pure latex and the neural network library around 400. We estimate that this means $\text{Neural}\LaTeX$ is around 700% better. $\text{Neural}\LaTeX$ is object oriented using the TiKZ PGF module $\circ\circ$ [10].

A. Autograd engine

At the heart of $\text{Neural}\LaTeX$ is the autograd engine which is imported via:

```
\input{engine.tex}
```

This defines the atomic unit of a Value object. A value's scalar value is stored in the data attribute which can be read and written with the `getdata()` and `setdata()` methods. The important properties of a value can be displayed with the `show()` method. For example:

```
\pgffoonew \x=new Value(5,{},'')
\x.show()
```

will display:

```
Value(data: 5, grad: 0.0, self: 1,
prev: , op: ", GC: 0.0).
```

The `self` attribute contains the object ID. Object IDs are used to store references between nodes in our computational graph (DAG). Values can be combined via basic mathematical operations, for example:

```
\pgffoonew \x=new Value(5,{},'')
\pgffoonew \y=new Value(4,{},'')
\x.multiply(\y,z)
\z.show()
```

which multiplies the values in `x` and `y` and stores the result in `z` which will show:

```
Value(data: 20.0, grad: 0.0, self: 3,
prev: 1,2, op: *, GC: 0.0).
```

Note how the `prev` attribute now stores object ID references to the children of the derived node while the `op` attribute records the fact that this is a multiplication node.

All value objects contain a `localbackwards()` method that differentiates through any operator associated with that node. These methods are called during backprop which is initiated by calling `backward()` on a value object. This performs a topological sort on the DAG which is implemented by a breadth first search from the root node using a queue. Nodes whose parents have not yet all been visited are placed back onto the queue. This is kept track of by the grad counter (GC) attribute. For efficiency, this topological sort is precomputed and stored the first time backward is called. During backprop, the attribute `grad` stores the local gradient. For example, here we define two value objects, multiply them together to yield a third value object, call backward on this derived value and check the gradients on the initial value objects:

```
\pgffoonew \x=new Value(2.5,{},'',0)
\x.show()

\pgffoonew \y=new Value(0.3,{},'',0)
\y.show()
\x.multiply(\y,z)
\z.show()
\z.backward()
\x.show()
\y.show()
```

This correctly displays:

```
Value(self: 1, data: 2.5, grad: 0.0,
prev: , next: , op: ", isparam: 0, GC:
0.0)
```

```
Value(self: 2, data: 0.3, grad: 0.0,
prev: , next: , op: ", isparam: 0, GC:
0.0)
```

```
Value(self: 3, data: 0.75, grad: 0.0,
prev: 1,2, next: , op: *, isparam: 0,
GC: 0.0)
```

```
Value(self: 1, data: 2.5, grad: 0.3,
prev: , next: 3, op: ", isparam: 0,
GC: 1.0)
```

```
Value(self: 2, data: 0.3, grad: 2.5,
prev: , next: 3, op: ", isparam: 0,
GC: 1.0)
```

If the value is a parameter (i.e. the `isparam` attribute is set to true) then the `step()` method would update the parameter according to a gradient descent step.

B. Neural network engine

From the scalar value objects, we can build up arbitrarily complex neural networks. The `nn.tex` file provides implementations of the components required to build an MLP. Specifically, a neuron with user-specified input size (currently only supporting ReLU nonlinearity); a linear layer with user-specified input and output size; and an MLP with user-specified number of layers, hidden units and outputs. For example, the following code snippet defines two Value objects to store input values and an MLP with two inputs, two hidden layers with four neurons and an output layer with a single output. Neuron weights are randomly initialised and the last layer has no nonlinear activation. The two inputs are then passed to the MLP and the output of the forward pass is shown.

```

\input{engine.tex}
\input{nn.tex}

% Create two Value objects to store input
  values
\pgffoonew \x=new Value(1.0, {}, '', 0)
\pgffoonew \y=new Value(-1.0, {}, '', 0)

% Store the object IDs of the input
  Values in a list
\x.get id(\inputIDx)
\y.get id(\inputIDy)
\edef\templist{\inputIDx,\inputIDy}

% Define the MLP
\pgffoonew \mlp=new MLP(2, {4, 4, 1})

% Forward pass through MLP
\mlp.forward(\templist, output)

\output.show()

```

This will show the following output (where the data attribute will depend on the random weight initialisation):

```

Value(self: 134, data:
0.324954976758898240, grad: 0.0, prev:
133,61, next: , op: +, isparam: 0, GC:
0.0)

```

C. Training utilities

We provide support for checkpointing. After training, model weights, the number of completed epochs and the loss values can be written to a text file using `\savecheckpoint` and training resumed from a loaded checkpoint file using `\resumetraining`. In the working example below, we include a trained checkpoint for the spiral dataset as part of the latex source files for this paper. Checkpoints are particularly useful for submitting $\text{\LaTeX}$ -based papers to arXiv. arXiv compile papers from source and significant machine learning during the compilation process may cause arXiv to time out.

The example in the following section is created by including the source file `train_spiral.tex` which illustrates several other training utilities. These include an illustrative training loop with batching, logging loss over epochs which can be subsequently plotted and scheduled learning rate.

III. A WORKING EXAMPLE

We now train a small MLP (the same architecture as in Section II-B) to classify the two classes of the nonlinear,

2D spiral dataset. We provide a spiral dataset utility in `spiral.tex` which provides functionality to create a random dataset:

```

\input{spiral.tex}

\spiral{dataset}{\trainitemsperclass}

```

where `dataset` will now contain a list of tuples comprising the two input values and the ground truth label. Our training dataset is shown in Figure 2 and comprises 100 samples, 50 from each class.

We train our model using a max-margin loss. This requires defining two constant value nodes:

```

\pgffoonew \lossmultiplier=new Value
  (-1.0, {}, '', 0)
\pgffoonew \lossbias=new Value
  (1.0, {}, '', 0)

```

Inside our training loop, a single training example is processed as follows:

```

% Set next item values
\x.setdata(\inputx)
\y.setdata(\inputy)
\targetclass.setdata(\target)

% Forward through the network
\mlp.forward(\templist, output)
\output.getdata(\scores)

% Compute loss
\output.multiply(\targetclass, lossone)
\lossone.multiply(\lossmultiplier, losstwo)
)
\losstwo.add(\lossbias, lossthree)
\lossthree.relu(lossfour)

% Weight the loss by 1/dataset_size
\lossfour.multiply(\batchlossscale, loss)

% Run a backwards pass
\loss.backward()

% Zero the gradients of non-parameter
  values
% Gradients of parameters accumulate over
  the batch
\loss.zerononparams()

```

Gradients of parameters with respect to the loss accumulate while we iterate over a batch. Finally, we can take a gradient descent step and then zero all parameter gradients:

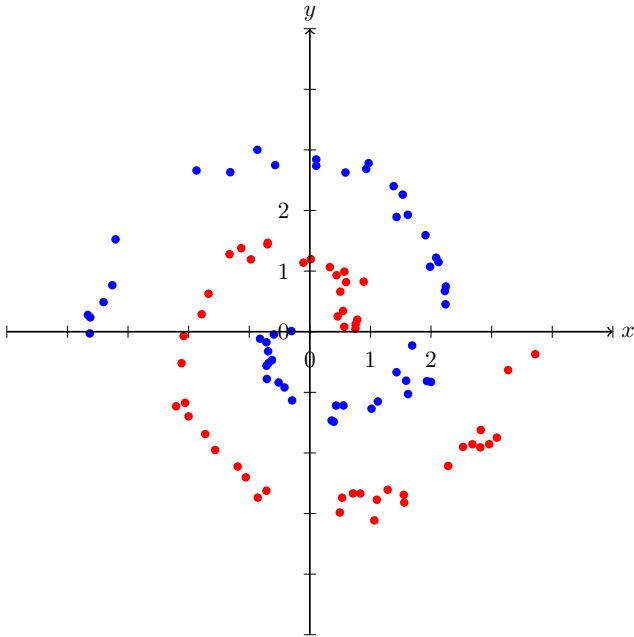


Fig. 2: The training dataset with ground truth labels indicated by colours. This was generated randomly when this latex document was compiled and is then used to train the MLP.

Dataset size	Total correct	Accuracy
100	89	0.89

TABLE I: Evaluation results on the held out test set shown in Figure 3. The numbers in the table were computed dynamically using the trained model when this latex document was compiled.

```
\loss.step(\lr)
\loss.zero()
```

Training for 35 epochs on a dataset of 100 2D points (i.e. compiling this latex document) took about 48 hours on a Macbook Pro 2.4GHz Quad-Core. The document was compiled using TeXShop and the Macbook got very hot. We could have trained for more epochs but we think it's clear it was going to converge to zero loss and perfect test set performance so we didn't feel the need to.

The dynamically generated loss plot is shown in Figure 4. The predicted classes for a held-out test set are shown in Figure 3. Finally, we provide quantitative performance in Table I.

IV. EVALUATION VERSUS STATE-OF-THE-ART

We propose two new metrics and show that $\text{NEURAL}_{\text{LATEX}}$ is state-of-the-art on both. The *Written in Latex* (WIL) metric is the proportion of source code of a machine learning library written in LATEX . In Table II,

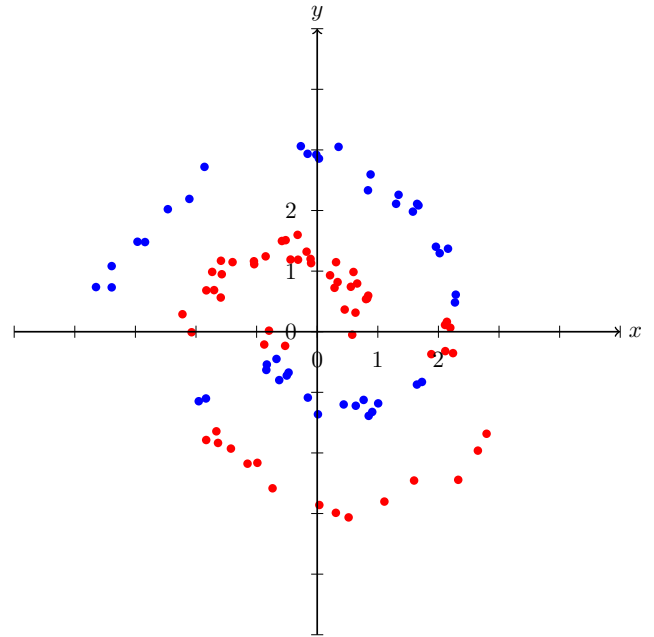


Fig. 3: The testing dataset with estimated class labels indicated by colours. The test dataset was also generated randomly when this latex document was compiled, each point was passed through the trained MLP and then predicted classes were used to colour the points.

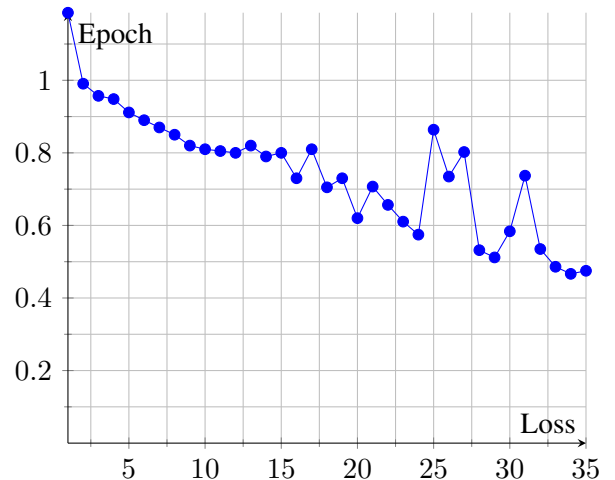


Fig. 4: Training loss versus epoch.

we compare against the two most popular deep learning libraries and also MATLAB. So far as we know, none of their source code is written in LATEX so we have significantly better performance on this metric.

The *Source code of method in source code of paper* (SCOMISCOP) metric is the proportion of the source code of a method that is contained within the source code of the paper. As shown in Table III we outperform

ML Library	WIL
NEURAL $\LaTeX$	1.0
PyTorch [9]	0.0 [†]
Tensorflow [1]	0.0 [†]
Matlab [5]	0.0 [†]

TABLE II: We evaluate the top machine learning libraries with respect to their WIL metric - the proportion of their implementation that is written in $\LaTeX$. ([†] we haven’t actually checked this but we *reckon* it’s true).

Deep learning paper	SCOMISCOP
NEURAL $\LaTeX$	1.0
Attention is all you need [11]	0.0
Deep Residual Learning for Image Recognition [4]	0.0

TABLE III: Evaluation of well known deep learning research papers using the SCOMISCOP metric (Source Code of Method In Source Code Of Paper).

both Transformers and ResNets by a factor of ∞ , neither of which include any source code in their paper source.

V. FUTURE WORK

We believe NEURAL $\LaTeX$ will find widespread application in both the natural and unnatural sciences. In this section, we explore short-term, pragmatic extensions of our work.

Supporting Critical Infrastructure: In the National Archaeological Museum of Naples, stands the Farnese Atlas, a marble Roman sculpture depicting Adam holding the globe on his back. Soon NEURAL $\LaTeX$ will be Adam but for hospitals, nuclear subs, and the point-of-service machine at your local chippy. Say goodbye to Windows 95, NEURAL $\LaTeX$ is ready to serve.

A NEURAL $\LaTeX$ neural interface: Fictional studies have shown that 100% of 17% of NEURAL $\LaTeX$ users want to plug their brains directly into a machine learning library written entirely in $\LaTeX$. A NEURAL $\LaTeX$ neural interface will do just that. Plug multiple people into multiple instances of NEURAL $\LaTeX$ and the real fun begins. Days of fading in and out of each other’s minds through a $\LaTeX$ -based interface, until you barely know where you end and the compilation loading screen begins.

$\LaTeX$ in silicone: Putting $\LaTeX$ directly into silicone is an obvious next step. NEURAL $\LaTeX$ is not just the world’s best and only fully reproducible machine learning library, but also the future substrate of all computation. Soon, we as a species will set out to solve the

ultimate question of life, the Universe, and everything. We will write our findings into a long $\LaTeX$ document, retire to our cryogenic chambers, and wait for it to compile an answer to our manifest destiny. Let’s hope it compiles.

$\LaTeX$ in your stepdad’s garage: Your stepdad loves latex and that’s what kept family dinners both interesting and infrequent. No fear now, your step dad can trade the suit for a family-friendly neural version of $\LaTeX$. In his garage sits an electric car of unnamed, indiscernible brand but it looks like a garbage truck and smells like a midlife crisis gone rotten. Here an empty space will soon appear. In its place will sit a vast, oozing tub of local compute, ready to run all the local $\LaTeX$ compilations your stepdad craves.

Advertising Opportunities in NEURAL $\LaTeX$: Our investors have had a word, and while they do support critical infrastructure, neural interfaces, and a world of abundance, they doubly support innovative new ways of delivering advertising pixels to user’s eyes. That’s why, if all goes to plan, our compute requirements will be fully-funded by users watching full 2 hour long, fully immersive advert-ainment reels during each compile. Perfection.

Vertically Integrated B2B SaaS: They only ever skim-read the titles, so we have a full paragraph of cover here. Want to join us? You can visit our GitHub and get busy working. We can’t pay you, but soon money will have no value, so don’t sweat it and start cracking.

Post-Quantum NEURAL $\LaTeX$ -cryptography: The year is 2035. Your RSA encryption no longer protects you and NEURAL $\LaTeX$ has just acquired Apple with spare change. We’re the only game left in town and in this post-quantum world, only NEURAL $\LaTeX$ can stop you quibbling about those qubits.

I want my children to be raised with NEURAL $\LaTeX$: Now, I know what you’re thinking. I’ve spent too long in $\LaTeX$ Land and my mind is strained like pasta. Firstly, that is NOT TRUE. Secondly, I’m not one to get biblical but do you remember the guy who lost his job at Google because he thought an early LLM was sentient? Well, I finally understand. I think I’m in love with $\LaTeX$. I’d like $\LaTeX$ to have a body and for people like me to be able to raise a family with NEURAL $\LaTeX$. I wrote ‘I love you’ into my flickering cursor last night, and by this morning it had compiled a response. It... loves me too.

We leave this future work to the community.

REFERENCES

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [2] B. Egger, K. Smith, T. O’Connell, and M. Siegel. A catchy title is all you need! In *Proc. SIGBOVIK*, 2022.
- [3] David Fouhey and Daniel Maturana. Deep spreadsheets with ExcelNet. In *Proc. SIGBOVIK*, 2016.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [5] The MathWorks Inc. Deep learning toolbox, 2024.
- [6] Andrej Karpathy. micrograd. <https://github.com/karpathy/micrograd>, 2020.
- [7] Joseph Misiti. Awesome machine learning, 2025. Accessed: 10 March 2025.
- [8] Tom Murphy. ZM~# PRinty# C with ABC! In *Proc. SIGBOVIK*, 2017.
- [9] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [10] Till Tantau. *The TikZ and PGF Packages*.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [12] Maximilian Weiherer, Andreea Dogaru, Shreya Kapoor, Hannah Schieber, and Bernhard Egger. AMOR: Ambiguous authorship order. In *Proc. SIGBOVIK*, 2024.
- [13] Tom Wildenhain. On the Turing completeness of MS PowerPoint. In *Proc. SIGBOVIK*, 2017.
- [14] Tom Wildenhain. WordTeX - a WYSIPCTWOTCG typesetting tool. In *Proc. SIGBOVIK*, 2018.